

Project 4: Bounded buffer Producer Consumer

Due by **October 29, 2024 11:59 pm**



Summary

In this project, we will build on the kernel module implemented in Project 3 to develop a fun and useful function in Linux, a **Zombie Killer**!

*It is the Halloween season, and zombies are roaming about. (Processes become zombies after they exit but their parents have not waited for them.) They are not only ugly but also causing sorts of damage to our world. (Zombie processes waste resources such as PIDs.) Unfortunately, no one is taking care of these crazy zombies. (Linux does not manage the zombie processes.) So it is up to you, **Linus the Zombie Killer** (here replace Linus with your name) to find and kill these zombies! (Hence the work of this project.)*

Description

We will use the kernel module from Project-3 to develop the Zombie Killer kernel module. The following describe the changes we need to make.

Module Parameters

To facilitate testing, we will use a test user to spawn zombie processes. Our kernel module will accept an additional argument of the test user's UID;

- **prod**: int, number of producer threads (a non-negative number)
- **cons**: int, number of consumer threads (a non-negative number)
- **size**: int, initializes the shared buffer(a non-negative number)
- **uid**: int, the UID of the test user

Shared Buffer

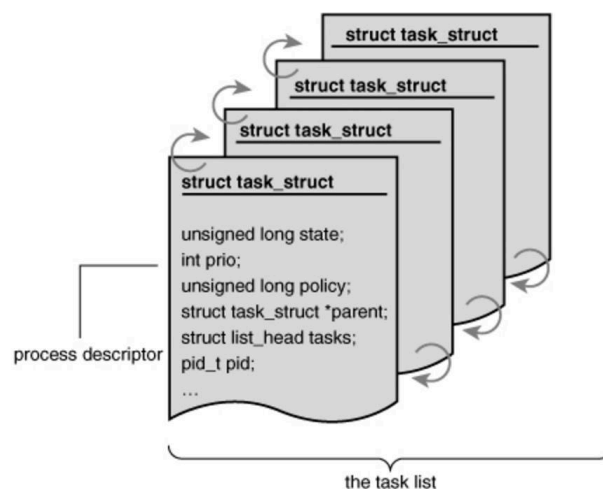
The producer thread and the consumer threads share a **bounded buffer** to store the **task_struct pointers** of the zombie processes. The size of this buffer is decided by the module parameter **size**. A single producer thread looks for zombie processes, and multiple consumer threads kill the zombie processes.

To make this buffer available to all the threads, we can declare it as a global variable in the module, and initialize it in `module_init` based on the parameter passed on to the module.

Finding Zombies

To simplify the project, we use only one producer thread, so **prod** is always set to 1. This single **producer** thread searches for all zombie processes that belong to the given test user, and for every zombie it finds, it stores the `task_struct` pointer of the process in the shared buffer.

The kernel stores the task structs of all the processes in a circular doubly linked list called task list, as shown in the following figure.



The kernel provides a convenient macro `for_each_process` for iterating the task list. The macro goes over all the task structs in the task list one by one.

```
for_each_process(struct task_struct *p)
```

The following code example calculates the number of processes in the task list that belong to the user UID.

```
#include <linux/sched.h>
#include <linux/sched/signal.h>

struct task_struct *p;
size_t process_counter = 0;
for_each_process(p) {
    /* Only focus on processes from specified uid */
```

```

    if (task->cred->uid.val != uid)
        continue;
    ++process_counter;
}

```

We can use the `exit_state` field of a process' `task_struct` to determine if the process is a zombie. To check if the `exit_state` indicates a zombie, we can use the `EXIT_ZOMBIE` bitmask (declared in `sched.h`) to perform the following bitwise operation on `exit_state`:

```

#define EXIT_ZOMBIE          0x00000020

```

```

#include <linux/sched.h>

if (ts->exit_state & EXIT_ZOMBIE) {
    // Handle the zombie process accordingly
}

```

The producer thread stores the `task_struct` pointer of a zombie process as an item into the shared buffer, and it prints the following into the kernel log:

[Producer-<producer-thread-id>] has produced a zombie process with pid <pid> and parent pid <pid>

Example: “[Producer-1] has produced a zombie process with pid 1337 and parent pid 1256”

Note: You **MUST** follow the above template to log messages.

To prevent the producer from using too much CPU time, we let it sleep a bit (**250ms** to be exact) after it iterates through the entire task list and before it starts the next iteration. We can accomplish this using the `msleep` function.

```

#include <linux/delay.h>
void msleep(unsigned int msecs);

```

Killing Zombies

The **consumer** threads retrieve the `task_struct` pointers from the shared buffer, and use them to clean up the zombie processes.

A zombie process is already dead, and therefore we cannot really kill it like a normal process. To get rid of the zombie process, we need to instead target the evil parent who spawned the zombie. To find out the parent process of a zombie, we can use the `parent` field of the zombie process' `task_struct`, and then use the `kill_pid` function to kill it:

```
#include <linux/sched/signal.h>
extern int kill_pid(ts->parent->thread_pid, SIGKILL, 0);
// ts is the child process here.
```

After killing a zombie process (by killing its parent), the consumer thread prints the following to the kernel log:

[Consumer-<consumer-thread-id>] has consumed a zombie process with pid <pid> and parent pid <ppid>

Example: “[Consumer-1] has consumed a zombie process with pid 1337 and parent pid 1256”

Note: You **MUST** follow the above template to log messages.

Requirements

- All the zombie processes should be identified by the producer thread.
- All the identified zombie processes should be killed by the consumer threads.
- All zombie processes should be identified and killed within a reasonable amount of time.
- No regular process should be either identified by the producer as a zombie or killed by any consumer thread.

Testing

- In order to test your project we have provided you with the grading scripts. They are available on a GitHub repository: <https://github.com/visa-lab/CSE330-OS.git>
- In order to clone the Github repository follow the below steps:
 - `git clone https://github.com/visa-lab/CSE330-OS.git`

- cd CSE330-OS/
- git checkout project-4
- Use [test_module.sh](#) to test your module code.
 - The script takes a path to your project directory (i.e., where you have your module source code), the value for **prod**, the value for **cons**, and the value for the **size** of the buffer, the number of **regular** processes to start, and the number of **zombies** to start.
 - The script will go into your project directory, compile your kernel module, load your kernel module, and verify the correctness of your implementation based on the provided module parameters.
 - The script will start both **regular** and **zombie** processes, and check how many zombies and regular processes remain after 10 seconds. The goal of your kernel module is to kill all the zombie processes and none of the regular processes.
 - Note that you may notice that your kernel module attempts to kill the shell that launches the parent processes of the zombie processes. This is because when your kernel module kills these parent processes, before the shell collects their exit status, there is a brief window during which their exit state is also EXIT_ZOMBIE. The testing script launches the shell as root instead of the test user so your kernel module should not be targeting it.
 - Below shows an example of the expected output from Test Case #3:

```

GTA-CSE33 0 > nvim - 1 > bash * 2 > bash 2024-10-17 < 23:53 vboxuser-VirtualB
vboxuser@vboxuser-VirtualBox:~/git/test/GTA-CSE330-Fall2024/Project4/testing$ ./test_module.sh .. 1 10 5 10 10
[log]: Creating user TestP4...
[log]: Look for Makefile
[log]: - file /home/vboxuser/git/test/GTA-CSE330-Fall2024/Project4/Makefile found
[log]: Look for source file (producer_consumer.c)
[log]: - file /home/vboxuser/git/test/GTA-CSE330-Fall2024/Project4/producer_consumer.c found
[log]: Compile the kernel module
[log]: - Compiled successfully
[log]: Starting 10 normal processes
[log]: Load the kernel module
[log]: - Loaded successfully
[log]: Starting zombie processes ...
[log]: - Total zombies spawned so far: 10/10
[log]: Checking the counts of the running kernel threads
[log]: - Found all expected threads
[log]: We will now wait some time to give your kernel module time to cleanup
[log]:   └─ We will wait 10 seconds
[log]: Checking the pids of all remaining processes against your output
[log]: - All zombies were cleaned
[log]: - None of the regular processes were killed
[log]:   └─ All zombies were cleaned
[log]:     └─ None of the regular processes were killed
[log]: Unload the kernel module
[log]: - Kernel module unloaded successfully
[log]: Checking to make sure kthreads are terminated
[log]: - All threads have been stopped
[zombie killer]: Passed
[final score]: 25.00/25
[log]: Deleting user TestP4...
vboxuser@vboxuser-VirtualBox:~/git/test/GTA-CSE330-Fall2024/Project4/testing$

```

- Use [test_zip_contents.sh](#) to validate that your zip file adheres to the directory structure that is expected. Below shows an example of the expected output:

```

GTA-CSE330 0 > nvim 1 > bash - 2 > bash *
vboxuser@vboxuser-VirtualBox:~/git/test/GTA-CSE330-Fall2024/Project4/testing$ ./test_zip_contents.sh /tmp/submit/Project-4-1225754101.zip
[log]: Look for directory (source_code)
[log]: - file /home/vboxuser/git/test/GTA-CSE330-Fall2024/Project4/testing/unzip_1729234456/source_code found
[log]: Look for Makefile
[log]: - file /home/vboxuser/git/test/GTA-CSE330-Fall2024/Project4/testing/unzip_1729234456/source_code/Makefile found
[log]: Look for source file (producer_consumer.c)
[log]: - file /home/vboxuser/git/test/GTA-CSE330-Fall2024/Project4/testing/unzip_1729234456/source_code/producer_consumer.c found
[test_zip_contents]: Passed
vboxuser@vboxuser-VirtualBox:~/git/test/GTA-CSE330-Fall2024/Project4/testing$

```

- **Note:** We will check your code in addition to running the test script, so passing the testing script does not guarantee getting all the grade points. But if your submission fails to pass the [test_zip_contents.sh](#) script, you will receive **zero** grade points.
- **Note:** We will use an anti-plagiarism tool and AI content detector to check the integrity of your code.

Submission Requirements & Guidelines

- Submit the following in a zip file to Canvas:
 - Source code: Make a directory named `'source_code'`, and include your code file named `'producer_consumer.c'` and your makefile named `'Makefile'`.
- You **MUST** name the zip file following the below naming convention: "Project-4-<Your ASU ID>.zip", e.g., Project-4-1225754101.zip
 - You can use the following command to create your zip file:
zip -r Project-4-1225754101.zip source_code/

Note: Extensions automatically generated by Canvas due to multiple submission attempts will be managed by the grading script.

- Do not submit any other source code
- Do not submit any binary
- If your submissions do not adhere to submission guidelines and naming conventions you will receive zero points.

Grading Rubrics

Test Case	Criteria	Points
-----------	----------	--------

Test Case #1 Test script starts 10 zombie processes for the test user.	Input: <code>./test_module.sh /path/to/code/ 1 10 5 0 10</code> Expected Output: 1. Thread count validated: prod=1, cons=10. 2. Kernel module killed 10 zombie processes within 10 sec after being loaded. 3. All kernel threads stopped after module unload.	2 pts thread count 20 pts 10 zombies are killed 3 pts all threads stopped ----- 25 points
Test Case #2 Test script starts 10 zombies and 10 regular processes for a test user.	Input: <code>./test_module.sh /path/to/code/ 1 10 5 10 10</code> Expected Output: 1. Thread count validated: prod=1, cons=10. 2. Kernel module killed 10 zombie processes and 0 regular processes within 10 sec after being loaded. 3. All kernel threads stopped after module unload.	2 pts thread count 20 pts 10 zombies are killed; 0 regular process killed 3 pts all threads stopped ----- 25 points
Test Case #3 Test script starts 100 regular and 100 zombie processes, with zombie processes started in batches of 10 processes for a test user.	Input: <code>./test_module.sh /path/to/code/ 1 20 10 100 100</code> Expected Output: 1. Thread count validated: prod=1, cons=20. 2. Kernel module killed 100 zombie processes and 0 regular processes within 10 sec after being loaded. 3. All kernel threads stopped after module unload.	2 pts thread count 20 pts 100 zombies are killed; 0 regular process killed 3 pts all threads stopped ----- 25 points

Test Case #4 Test script starts 1000 regular and 1000 zombie processes, with the zombie processes started in batches of 100 processes for a test user.	Input: <pre>./test_module.sh /path/to/code/ 1 50 100 1000 1000</pre> Expected Output: 1. Thread count validated: prod=1, cons=50. 2. Kernel module killed 1000 zombie processes and 0 regular processes within 10 sec after being loaded. 3. All kernel threads stopped after module unload.	2 pts thread count 20 pts 1000 zombies are killed; 0 regular process killed 3 pts all threads stopped ----- 25 points
Submission Content	<pre>./test_zip_contents.sh /path/to/zip/file.zip</pre> 1. Zip file should contain only raw outputs (screenshot), source code. No binaries. 2. All files adhere to the directory structure specified in the project spec. Failure to adhere to the directory structure will result in zero.	-5 pts unwanted files or binaries

Policies

- Late submissions will **absolutely not** be graded (unless you have verifiable proof of emergency). It is much better to submit partial work on time and get partial credit for your work than to submit late for no credit.
- Every student needs to **work independently** on this exercise. We encourage high-level discussions among students to help each other understand the concepts and principles. However, a code-level discussion is prohibited and plagiarism will directly lead to failure of this course. We will use anti-plagiarism tools to detect violations of this policy.
- **The use of generative AI tools is not allowed** to complete any portion of the assignment.

